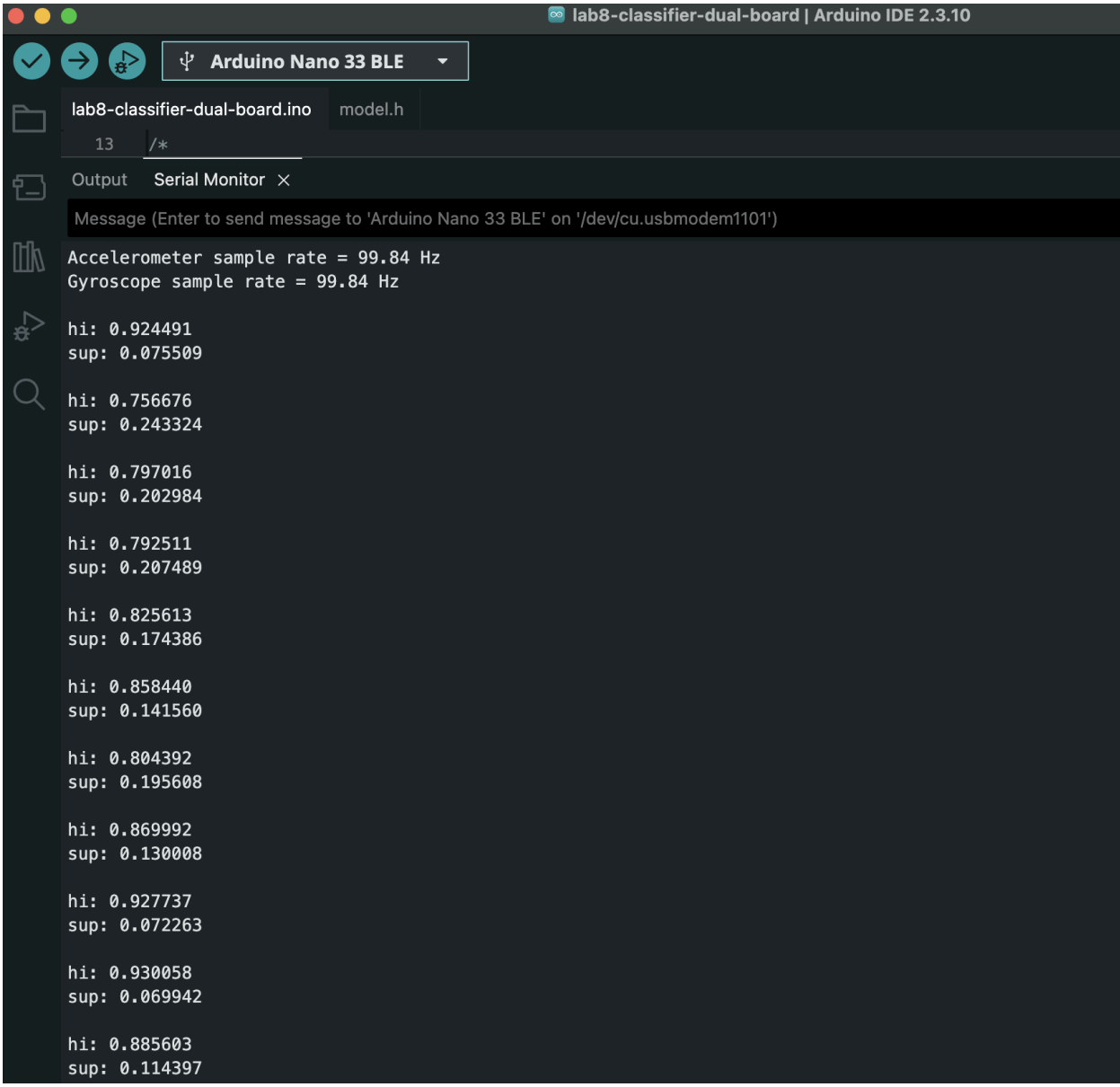
Lab 8 Report

Basic Information

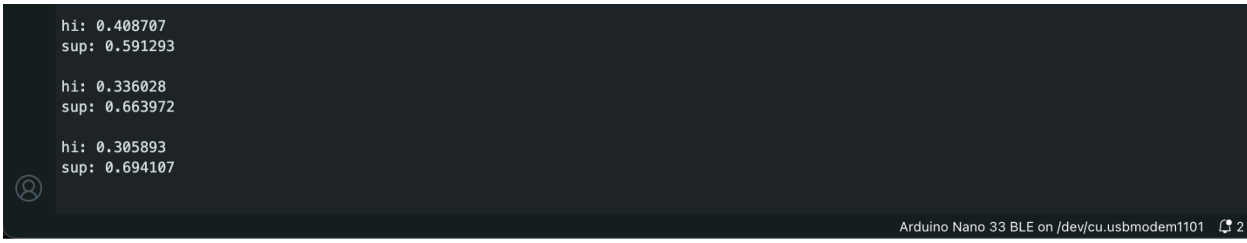
- Name: Richard (Yunchao) He
- Report document: https://docs.google.com/document/d/1kREgQaE-70tBFT-3lLI7qZtRUuWOk42jppuSpoNnYaI/edit

Screenshots

- Screenshots showing both ASL gestures correctly identified by the Arduino Serial Monitor.
    - In my experiment, the ASL gesture “Hi” is much easier to be recognized by the model. And the confidence score is very high by softmax, which is above 0.9 sometimes.



- However, “Sup” (what’s up) is not that easy to be recognized. The highest confidence score by the model (from my deployment data only) is around 0.69 by softmax.



Software Issue

Software issue fixed

- A brief explanation of the software issue I fixed in the notebook
    - The original model was compiled using mean squared error (`mse`) as the loss function and mean absolute error (`mae`) as the evaluation metric. These settings are more appropriate for regression tasks, while this lab performs categorical classification with one-hot encoded labels and a softmax output layer. I changed the loss function to `categorical_crossentropy`and the evaluation metric to `accuracy`.
    - The code changes is actually tiny (marked in a blue rectangle in the figure below)

```
[6]: # Build the model and train it.
model = tf.keras.Sequential()
model.add(tf.keras.layers.Dense(50, activation='relu'))  # ReLU is used for performance.
model.add(tf.keras.layers.Dense(15, activation='relu'))
model.add(tf.keras.layers.Dense(NUM_GESTURES, activation='softmax'))  # Softmax is used because only one gesture is expected per input.
# model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])

model.compile(
    optimizer='rmsprop',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    inputs_train,
    outputs_train,
    epochs=100,
    batch_size=16,
    validation_data=(inputs_validate, outputs_validate),
)

print("History keys:", list(history.history.keys()))
```

Downstream consistency changes

- A brief explanation of the downstream consistency changes
    - In order to keep the downstream evaluation and plots consistent with the corrected training configuration, I changed METRIC_TO_PLOT from mae to accuracy. As a result, the notebook now reads the accuracy and val_accuracy history values and labels the training plot accordingly.
    - The code change is pretty small, see the blue rectangle in the figure below

[9]:
```
# METRIC_TO_PLOT = "mae"  # Update this if you changed the metric used during training.
METRIC_TO_PLOT = "accuracy"
VAL_METRIC_TO_PLOT = f"val_{METRIC_TO_PLOT}"

metric_values = history.history[METRIC_TO_PLOT]
val_metric_values = history.history[VAL_METRIC_TO_PLOT]

plt.figure(figsize=(12, 5))
plt.plot(epochs[SKIP:], metric_values[SKIP:], label=f"Training {METRIC_TO_PLOT}")
plt.plot(epochs[SKIP:], val_metric_values[SKIP:], label=f"Validation {METRIC_TO_PLOT}")
plt.title(f"Training and Validation {METRIC_TO_PLOT}")
plt.xlabel("Epoch")
plt.ylabel(METRIC_TO_PLOT)
plt.legend()
plt.grid(True)
plt.show()
```

Other change

The lab mentioned **quantizing** the model for tinyML. So, I attempted post-training optimization using `tf.lite.Optimize.DEFAULT`, which reduced the model size from 148,296 bytes to 41,296 bytes via **INT8 quantization for model parameters** (not including input/output data). However, the resulting hybrid model was not supported by the TensorFlow Lite Micro fully connected kernel on the Arduino. Although compilation and upload succeeded, model initialization failed during `AllocateTensors()`. I therefore reverted to the FP32 TFLite model, regenerated `model.h`, and successfully deployed and tested it.

See the commented code snippet below in blue rectangle:

[12]:
```
converter = tf.lite.TFLiteConverter.from_keras_model(model)

# Enable TensorFlow Lite model optimization and quantization.
# converter.optimizations = [tf.lite.Optimize.DEFAULT]

tflite_model = converter.convert()

tflite_path = BUILD_DIR / "gesture_model.tflite"
tflite_path.write_bytes(tflite_model)

print(f"Saved: {tflite_path}")
print(f"TFLite model size: {tflite_path.stat().st_size:,} bytes")
```